and store the result. In all other cases, like for `search`, we simply know that we can't get anywhere.

#8 Now check the result of the (possible) recursive call: if it found something, just return it.

#9 Or, if this is a key node, the longest possible prefix in the trie could be the path to current node (accumulated in `prefix`).

#10 Otherwise, we know that we haven't find any result in this subtree, and return `null` to let the caller know.

6.3.6 Keys Starting with a Prefix

For tries, this method leverages search to find the starting point of a full-fledged traversal, retrieving all keys in the subtree rooted at the prefix.

Unfortunately for radix tries the situation is a bit more complicated, because prefixes that are not stored in the tree can partially match edges: take, for instance, the tree in figure 6.20, where prefixes like "`a`" or "`anth`" are not stored in the tree, the latter doesn't even have a node at the end of the corresponding path, but still the radix trie contains several words starting with those prefixes.

If we were just looking for nodes that lie at the end of the path for those strings, we would miss all those legit results. We need, instead, to rewrite a special version of the search method for this operation, where we distinguish the edge match case #2, where we have a no-go, and case #3, where, instead, since the string fragment searched is a proper prefix of the last edge in the path, the subtree referenced by the edge will indeed contain strings that match the searched